

错开了，为什么还会等？

一个取料窗口、两位厨师，只用整数讲明白

通俗图解·所有数字均为教学虚构

1 先看一次：已经错开，为什么还是停工？

小林和老周各自在灶台做菜，共用一个备料窗口。窗口每秒装 1 袋，谁先排单就先装谁的，不插队。每袋一样大；老周要 6 袋，就是连续排好 6 张小单，不是一袋特别大的材料。

开做当前菜时，厨师就下单下一道菜的材料。排队的是订单，厨师本人继续做菜。下一份料必须在当前菜做完时到齐，否则厨师才真正停工。只允许提前准备下一道菜，配齐后立即送到厨师手上。

对应到电脑：厨师是 NPU，做菜是计算，窗口装袋是 SSD 读取，订单队伍是 Path0。这里只用故事解释机制，不代表真实模型的性能。

1.1 老周先来，小林晚 1 秒：不够

看已经开工后的某一轮：老周第 0 秒排好 6 袋，窗口要装到第 6 秒。小林第 1 秒开做当前菜，同时下单 1 袋。他当前菜要做 4 秒，所以第 5 秒就需要下一份料。

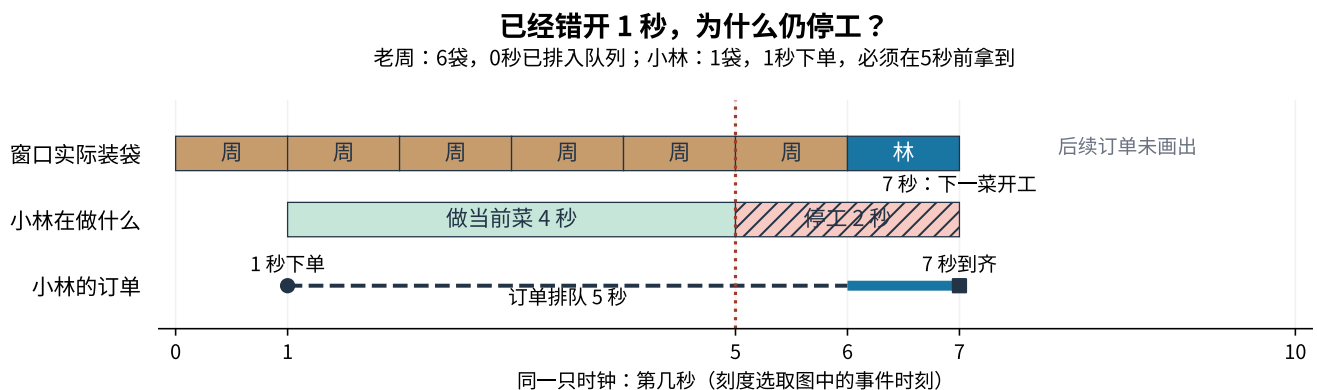


图 1：先读三行：窗口在忙谁（每格 1 袋）；小林在做菜还是停工；小林订单从下单到到齐。虚线表示订单排队，不是窗口在给小林装袋。红色竖虚线是必须拿到材料的时刻。

小林的订单第 1-6 秒排队，第 6-7 秒装袋，第 7 秒才到齐。他第 5 秒已经做完当前菜，只能停到第 7 秒：**停工 2 秒。**

问题不是“有没有错开”——已经错开 1 秒。问题是：前面还要 5 秒，自己再要 1 秒，可手头的菜只能撑 4 秒。

2 怎样才算错开有效？再看两个只改一个条件的对照

2.1 仍然做菜 4 秒，改成晚 3 秒开始：这次赶上

现在小林第 3 秒才开始当前菜，也在第 3 秒下单。他的下一份料仍是第 7 秒到齐，而当前菜恰好也做到第 7 秒。于是：**排队 3 秒 + 装袋 1 秒 = 能等的 4 秒，不停工。**

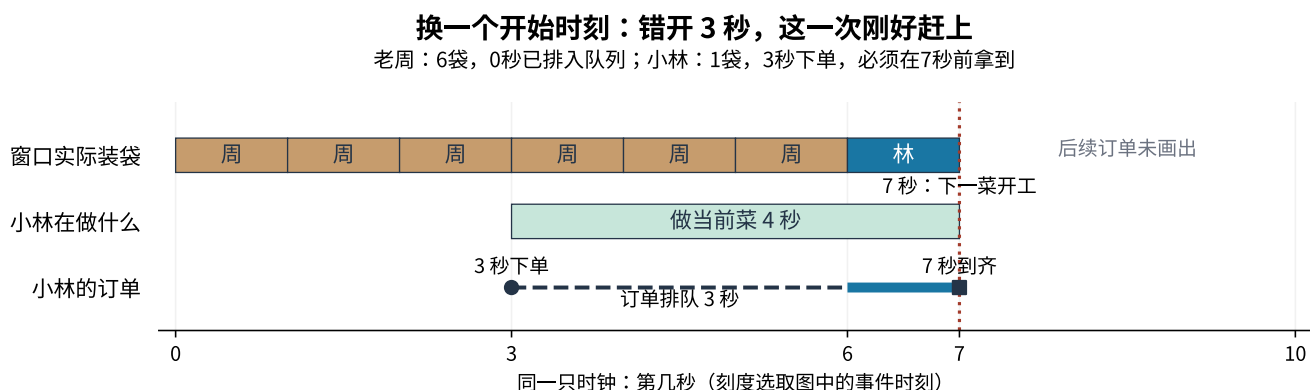


图 2：与图 1 相比，老周的 6 袋和窗口装袋时段完全不变。小林整道当前菜的开始时刻后移了，因此需料时刻也从第 5 秒移到第 7 秒。

注意：不是把“下单”偷偷拖晚、却仍要求第 5 秒拿到。这是两位厨师原本就处于不同做菜进度的一次对照。它证明这种相位能让这份材料及时，不证明系统会自动选中这个相位。

2.2 仍晚 1 秒开始，但当前菜要做 8 秒：也赶上

小林仍在第 1 秒下单，仍第 7 秒拿到；只是手头这道菜要做到第 9 秒。材料比需要时早到 2 秒，**订单虽然等了 6 秒，厨师却没有停工。**

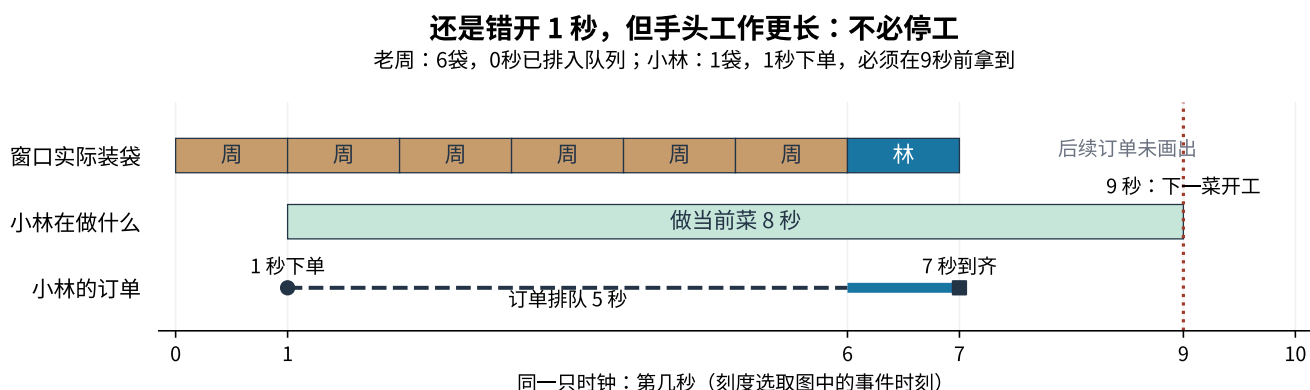


图 3：与图 1 相比，排队和装袋时间都没变；只改变小林当前菜的工作时间。菜的烹调时长不必与材料袋数成比例。

这时小林仍要到第 9 秒才开始下一道菜、下下一张单，不会因为第 7 秒材料到了就提前开始。**能被手头工作覆盖的等待，不会推迟下一次开工。**

3 你的想法有时确实成立：第一次拥挤，后来自然错开

换一个更轻的厨房：小林和老周每道菜都只要 1 袋，都要做 4 秒。窗口仍然每秒装 1 袋，所有规则不变。这次从两人第一次开工前讲起。

第 0 秒，两人都来要第一道菜的材料，小林先排到。窗口第 0-1 秒 给小林装袋，第 1-2 秒 给老周装袋。

小林第 1 秒 就能开工，老周第 2 秒 才能开工。首次取料，自然制造出了 1 秒的错位。两人开工时，都顺手下单下一道菜的材料。

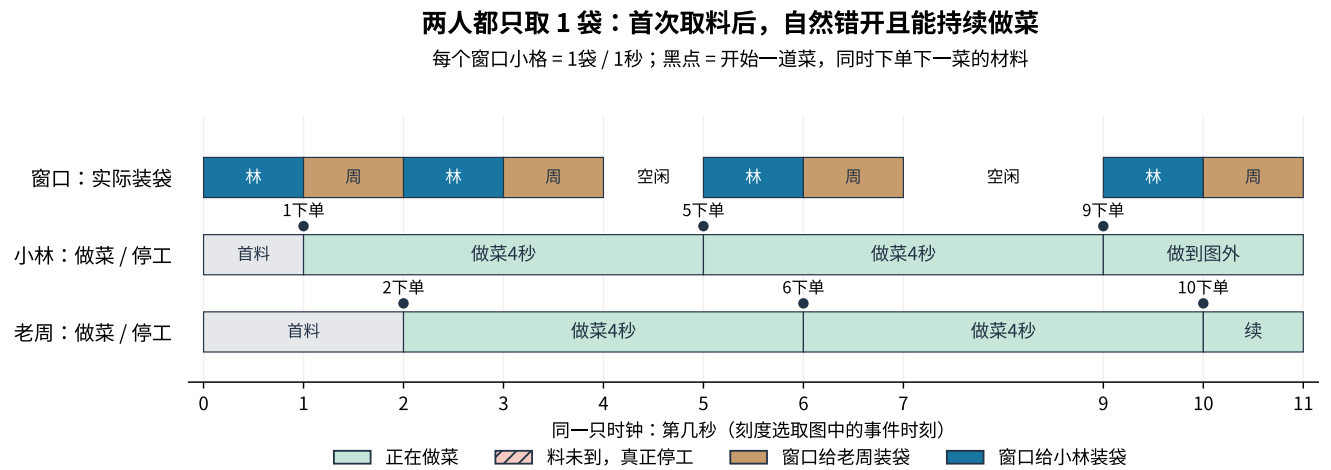


图 4：所有横条按真实时间排列。窗口每格是一袋；浅绿色是真正做菜，黑点是开始一道菜并下单下一菜。左侧灰色是首次备料。右侧只画到第 11 秒，两人仍在做菜，并非做菜时间缩短了。

先逐项核对第一次预取，不靠“看起来错开了”判断：

下一道菜的材料	小林	老周
下单时刻	第 1 秒	第 2 秒
窗口装袋	第 2-3 秒	第 3-4 秒
材料到齐	第 3 秒	第 4 秒
当前菜做完，必须用料	第 5 秒	第 6 秒
是否停工	否	否

小林这张单为什么不是第 1-2 秒就装？因为老周在第 0 秒提交的第一张单已排在前面，小林的新单不能排队。**这一轮仍有排队，只是两人都等得起。**

后来，小林在第 5、9 秒 开工并下单，老周在第 6、10 秒 开工并下单。每次窗口各用 1 秒处理，都能赶上各自 4 秒后的需料时刻。

所以，这次“自然错开”成功的完整原因是：首次取料把开工时刻分开，而错开后的等待加装袋时间，确实小于各自的 4 秒预算。

4 但等待不是自动找空档：看一个会重复的反例

现在小林仍是 1 袋、做菜 4 秒；只把老周改成 6 袋、做菜 10 秒。仍然每人仅提前准备下一道菜，窗口先来先服务。

下面画两人运行一阵后的一段，时钟重新记为第 0 秒。这不是又回到冷启动；图最左边，小林还在完成上一道菜的最后 1 秒。这个时序已用独立玩具程序从冷启动算出，并非随意拼出来。

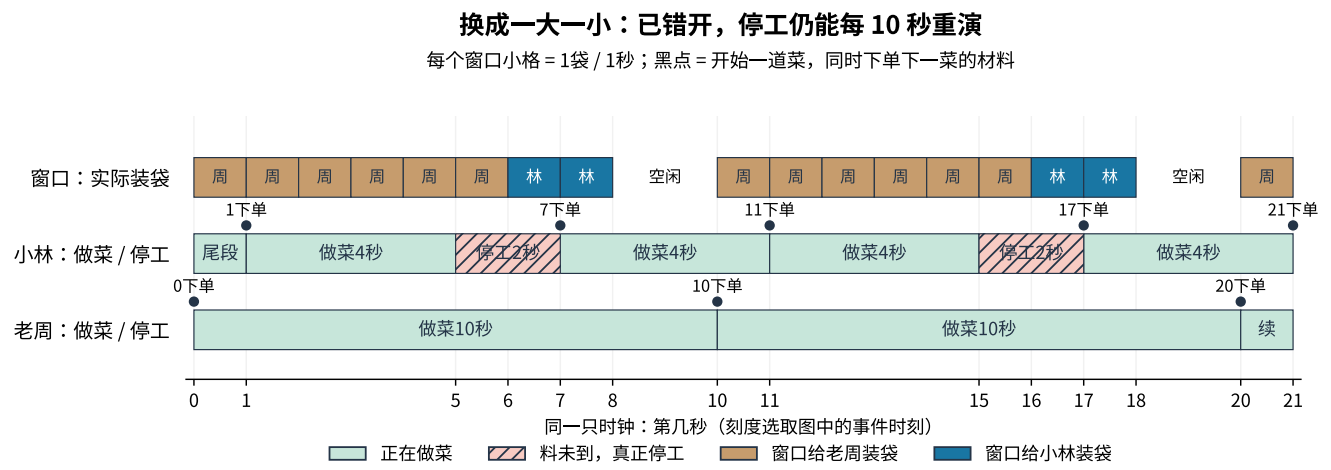


图 5：老周第 0、10 秒下单，小林第 1、11 秒下单：已有错位，仍在第 5-7 秒、第 15-17 秒停工。窗口第 8-10 秒、第 18-20 秒真正空闲。第 21 秒只是截图边界，未画出的后续工作还在继续。

跟着小林走一遍，就能看出“等过之后，怎么又遇上了”：

时刻	小林做了什么	结果
第 1 秒	开工，给下一菜下单	老周还有 5 袋在前面
第 5 秒	当前菜做好	材料还没齐，开始停工
第 7 秒	收料开工，再下下一单	这张新单第 8 秒就到齐
第 11 秒	开工，给下一菜下单	老周第 10 秒又排好了 6 袋
第 15-17 秒	当前菜已做好，下一份料没齐	又停工 2 秒

小林确实因为等待改变了时刻：他的下单是 1 → 7 → 11 → 17 → 21 秒，不是始终每隔 4 秒。但每两道菜花去“做 4 秒 + 停 2 秒 + 再做 4 秒”，一共 10 秒；老周也恰好每 10 秒下一次单。于是小林又回到了“晚老周 1 秒”的位置。

等待会改变时刻，但它没有“挑一个以后都不堵的位置”的能力。本例不是没错开，而是错开后的坏位置可以重复。

5 关键因素到底是什么？记住这三个问题

5.1 第一：前面的活，还剩多少？

不是只问“我晚来了几秒”，而是问“我来时，前面还要忙几秒”。老周 6 袋要装 6 秒，小林晚来 1 秒时，前面还剩 5 秒；晚来 3 秒时，只剩 3 秒。窗口更快、单子更小、前面队伍更短，都能减少这部分等待。

5.2 第二：我自己要多久，手头工作又能撑多久？

判断这一次是否会停工，只要比较：

前方剩余时间 + 自己装袋时间

材料到齐需要多久

≤

当前菜还能做多久

我能等多久

.

同一个 6 袋大单	我来时前方剩余	自己装袋	能等多久	停不停
晚 1 秒，小林做菜 4 秒	5 秒	1 秒	4 秒	停 2 秒
晚 3 秒，小林做菜 4 秒	3 秒	1 秒	4 秒	不停
晚 1 秒，小林做菜 8 秒	5 秒	1 秒	8 秒	不停

排队不等于停工；错开不等于及时。只有前两项之和超过能等的时间，多出来的那部分才是真正停工。

5.3 第三：下一轮还会不会回到坏位置？

这取决于大家做菜多久、每次取多少、什么时候下下一单，以及能提前备几道菜。上一页给出了一个重复反例，**不是说任何工作量、任何错位都会失败**。同样，一次成功也不能保证下一次一定成功。

5.3.1 你可能还会问：窗口明明有空，为什么还停？

反例即使两人都不停工，每 20 秒也只想要“小林 5 袋 + 老周 12 袋”，一共 17 袋，低于窗口最多能装的 20 袋。它不是平均能力不足。

实际循环里，小林有时来得急、前面又太长，就先停工；等到后面的空档，他还在做当前菜，而且只允许预取下一道菜，不能自动把以后十道菜都备好。按图中的第 1-21 秒计：小林做菜 16 秒、停 4 秒；老周做菜 20 秒；窗口只忙 16 秒、空 4 秒。这些都是时间轴上的真实区间之和，不是把等待搬到末尾画出来。

改成更早预取、更多窗口或允许紧急小单插队，可能改善结果，但那是在**改变机制**：需要更多材料暂存位置、更多资源，或让别的订单晚一些，不能归功于“自然错开”。

最后一句：错开有没有用，要看它能不能让“前面的活 + 我的活”在“我等得起的时间”内完成；是否能一直有效，还要看下一轮会不会重演坏位置。